

ComicCraft: AI-Powered Comic Story Creator

Project Description:

ComicCraft harnesses Generative AI to turn a short story idea into a complete five-panel comic. The user enters a story prompt, names a main character, and chooses a setting, a story tone and an art style. The platform then writes a structured story with a title, scene description, narration and dialogue for each panel, creates a matching image for every panel, and lets the user download the finished comic as a PDF.

Using Google's Gemini models for both story writing and image generation, ComicCraft removes the need for drawing skills and design software. The application is built with Python and served with Uvicorn, with a clean HTML and CSS interface. With secure API key management through a .env file and input handling in the backend, ComicCraft helps students, teachers and content creators bring their stories to life quickly and with confidence.

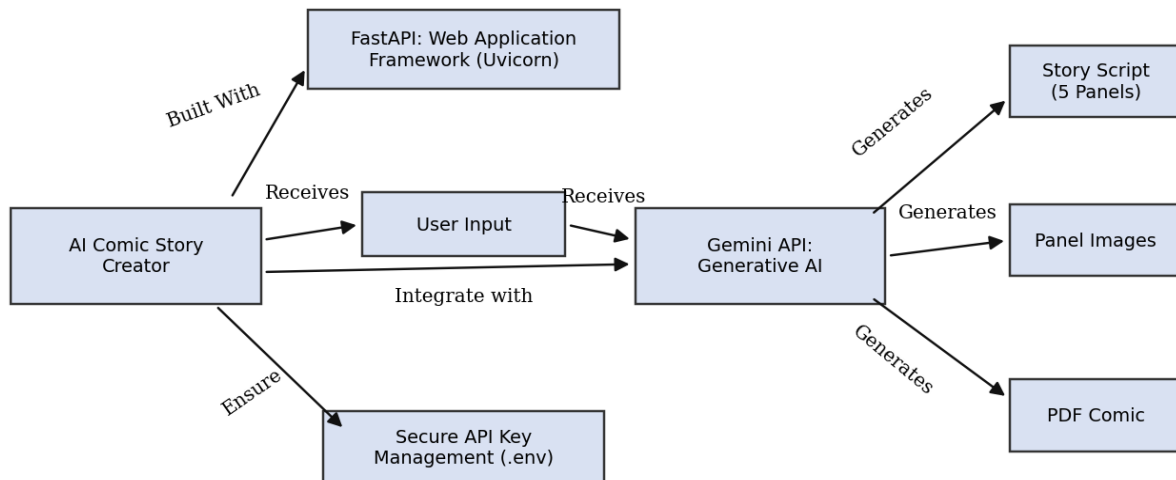
Scenarios

Scenario 1: A user types the story idea "A young programmer discovers a mysterious AI robot that can bring drawings to life", names the main character Maya, and selects a Space station setting, a Whimsical tone and a Watercolor comic art style. ComicCraft produces a five-panel story with a watercolor image, narration and dialogue for each panel, ready to download as a PDF.

Scenario 2: A teacher preparing a lesson enters a short story about a lesson topic and picks a friendly tone. ComicCraft creates an illustrated comic that can be shared with the class, saving the time normally needed to write the story and find or draw the pictures.

Scenario 3: A student with strong story ideas but no drawing skills enters a prompt for a class project, chooses an art style, and downloads the finished comic as a PDF to submit or share with friends.

Technical Architecture:



Description: ComicCraft uses a modular three-tier architecture to deliver AI-driven comic creation. The frontend provides a responsive web interface built with HTML and CSS, where the user fills in the story form and views the generated comic. The Python backend, served by Uvicorn, validates the user's input, builds the prompts and coordinates the calls to the Gemini API: first to write the five-panel story script, then to generate an image for each panel. The finished panels are assembled into a PDF that the user can download. The Gemini API key is kept out of the source code in a .env file.

Note: ComicCraft does not use a database, so an ER diagram is not required. The story and images are created for each request and the final comic is exported as a PDF file.

Pre-requisites:

- **Python Programming Proficiency:**

[Python Documentation](#)

- **FastAPI and Uvicorn Knowledge:**

[FastAPI Documentation](#)

- **Gemini API Familiarity:**

[Gemini API Documentation](#)

- **HTML and CSS Skills:**

[W3Schools HTML/CSS Tutorials](#)

- **Version Control with Git:**

[Git Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate Gemini text and image models for story and comic panel generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend and AI integration.
- **Activity 1.4:** Set up the development environment, installing the necessary libraries and dependencies.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: story generation, panel image generation and PDF export.
- **Activity 2.2:** Implement the backend routes to manage user input, comic generation and file download.

Milestone 3: Main Application Development

- **Activity 3.1:** Write the main application logic in the comiccraft.main module, establishing the routes for each page and integrating the AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML and CSS, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create the home, comic result and export success pages that display content based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the environment and installing all dependencies.
- **Activity 5.2:** Run the application with Uvicorn and test the complete flow in the browser.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI models from Google Gemini for our comic creation needs. This involves understanding what the Gemini text and image models can do, ensuring that the chosen models fit the application's objective of turning a short story idea into a five-panel illustrated comic.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with Gemini, you need a valid API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Open Google AI Studio:** Visit <https://aistudio.google.com> and sign in with your Google account.
- **Step 2 — Go to the API Keys page:** Open the API keys section from the menu.
- **Step 3 — Create a New API Key:** Click the "Create API key" button and select or create a project for the key.
- **Step 4 — Copy the Key:** Copy the key immediately and store it in a safe place.
- **Step 5 — Add the Key to Your Project:** Copy the `.env.example` file to a new file named `.env` in the project root and paste your key into it, as shown in `.env.example`.
- **Important — Never Commit Your API Key:** Make sure `.env` is listed in your `.gitignore` file so the key is never pushed to a public repository.

Activity 1.2: Research and Select the Appropriate Generative AI Models

Here are the things that need to be researched to select the best models for the project:

- **Understand the Project Requirements:** Review the needs of ComicCraft, which must write a five-panel story (title, scene description, narration and dialogue) and create a matching image for each panel.
- **Explore the Gemini Documentation:** Visit the Gemini API documentation to examine the available text and image models, their features, speed and limitations.
- **Evaluate Model Performance:** Compare the models for story quality, image quality, generation speed and how well they follow the chosen tone and art style.
- **Select the Models:** Choose a Gemini text model for writing the story script and a Gemini image model for creating the panel images.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application, including the frontend (HTML and CSS), the Python backend served by Uvicorn, and the Gemini API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application: entering a story prompt, naming the main character, and choosing a setting, story tone and art style.
- **Outline Backend Responsibilities:** Specify how the backend receives the form data, validates it, builds the prompts and communicates with the Gemini API.
- **Describe AI Integration Points:** Define how the story script is requested first, how each panel's image prompt is then used to generate its image, and how the results are passed to the result page and the PDF export.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3 is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
comic-craft> python -m venv .venv  
comic-craft> .venv\Scripts\activate
```

- **Install the Dependencies:** Install the project and its libraries from the pyproject.toml file:

```
(.venv) comic-craft> pip install -e .
```

- **Configure the API Key:** Copy .env.example to .env and add your Gemini API key.
- **Set Up Application Structure:** Organize the project into the following files and folders:

```
src/ - Source code of the comiccraft application  
tests/ - Automated test files  
build/ - Build output  
.env.example - Example environment configuration  
.gitignore - Files excluded from the repository  
pyproject.toml - Project settings and dependencies  
README.md - Setup and run instructions
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Comic Generation Features

Implement the three core features of ComicCraft: the story generator, the panel image generator and the PDF exporter.

- **Story Generator:** Takes the story prompt, main character, setting, story tone and art style and asks the Gemini text model for a five-panel story. Each panel has a title, a scene description, narration, dialogue and an image prompt.
- **Panel Image Generator:** Uses each panel's image prompt, together with the chosen art style, to create a picture for that panel with the Gemini image model.
- **PDF Exporter:** Combines the panel images and text into a single PDF file that the user can download and share.

Activity 2.2: Implement the Backend Routes

Define Routes:

- **Home page (/):** Shows the story form where the user describes the comic.
- **Generate (/generate):** Receives the form data, creates the story and images, and shows the five-panel comic.
- **Export success (/export-success):** Confirms that the comic PDF was created and starts the download.
- **Download (/download-comic/<filename>):** Sends the generated PDF file to the user.

Process User Input:

- Create a form on the home page for the story prompt, main character, setting, story tone and art style.
- Check that the story prompt is provided before processing.

Integrate API Calls:

- Build the prompts from the user's choices and send them to the Gemini text model to get the story script.
- Send each panel's image prompt to the Gemini image model to create the panel image.
- Show a clear error message if generation fails, so the user can try again.

Milestone 3: Main Application Development

Milestone 3 focuses on creating and refining the main logic of the application, which lives in the `comiccraft.main` module and serves as the backbone of ComicCraft. In this milestone, you set up the routes for each page, connect them to the story, image and PDF steps, and test each part to make sure it works as expected.

Activity 3.1: Writing the Main Application Logic

Define the Core Routes:

- Create the routes for the home page, comic generation, export success and download, each serving a distinct purpose.

Set Up the Generation Flow:

- Read the story prompt, main character, setting, story tone and art style from the submitted form.
- Request the five-panel story from the Gemini text model and generate each panel's image with the Gemini image model.
- Pass the finished panels to the result page, which shows each panel with its image, description, narration and dialogue.

Handle the PDF Export:

- Create the PDF from the generated panels and save it with a file name based on the story, for example `Maya_Discovery_2.pdf`.
- Redirect the user to the export success page, where the download starts and the user can download again or create another comic.

Test the Application:

- Run the automated tests in the tests folder and try different story prompts, tones and art styles to check that the output is generated correctly.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a friendly and attractive interface for ComicCraft. This involves designing a clean, light-purple layout with responsive HTML and CSS so that the story form, the generated comic and the export page are easy to use on different screen sizes.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main pages with a header showing the ComicCraft logo, a main content area and a footer.
- Use semantic HTML elements to keep the structure organized and accessible.

Design a Responsive Layout Using CSS:

- Create a stylesheet with rounded cards, soft shadows and a purple colour theme.
- Use flexbox and grid layouts to organize the content, for example the story form beside a preview card, and the five comic panels in a grid of cards.
- Add media queries so the layout adapts across desktop, tablet and mobile screen sizes.

Create Separate UI Components:

- **Story Form:** Fields for the story prompt, main character, setting, story tone and art style, with a Generate comic button.
- **Panel Cards:** One card per panel showing the panel number and title, the image, a short description, narration, dialogue and an expandable image prompt reference.
- **Action Buttons:** Download PDF and Create another comic buttons on the result and export pages.

Activity 4.2: Showing Content Based on User Actions

- After the user submits the form, show the generated comic page with the title, the five panels and the action buttons.
- After the PDF is created, show the export success page with the file name and the download buttons.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the ComicCraft application locally and verifying that the complete flow works correctly. This involves configuring the environment, installing the dependencies and launching the application.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a new terminal in the project folder.
- Create and activate a virtual environment, then install the dependencies:

```
comic-craft> python -m venv .venv  
comic-craft> .venv\Scripts\activate  
(.venv) comic-craft> pip install -e .
```

Configure Environment Variables:

- Copy .env.example to .env and add your Gemini API key. The application reads it from the .env file when it starts.

Activity 5.2: Local Testing and Verification

Start the Application:

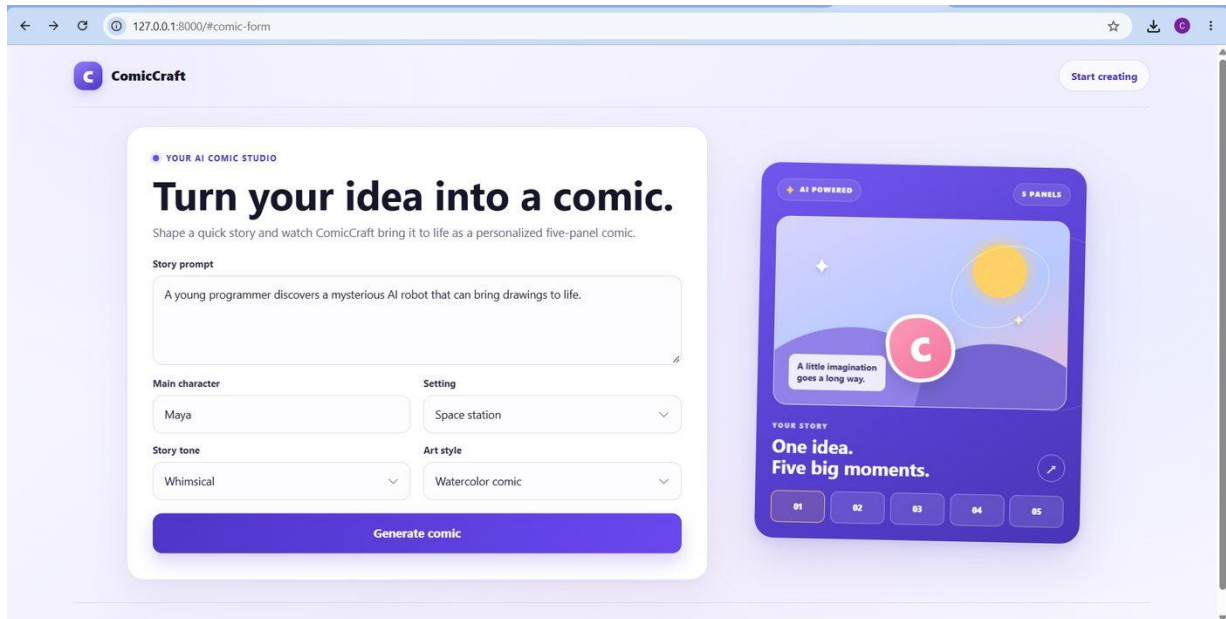
- Run the application locally using Uvicorn:

```
(.venv) comic-craft> uvicorn comiccraft.main:app --reload
```

- Once the terminal shows that application startup is complete, open a browser and go to <http://127.0.0.1:8000> to access the app.
- Enter a story idea, choose the setting, tone and art style, and click Generate comic. Check that the five panels, the narration and the dialogue are created, and that the PDF downloads correctly.

Exploring the Website's Web Pages:

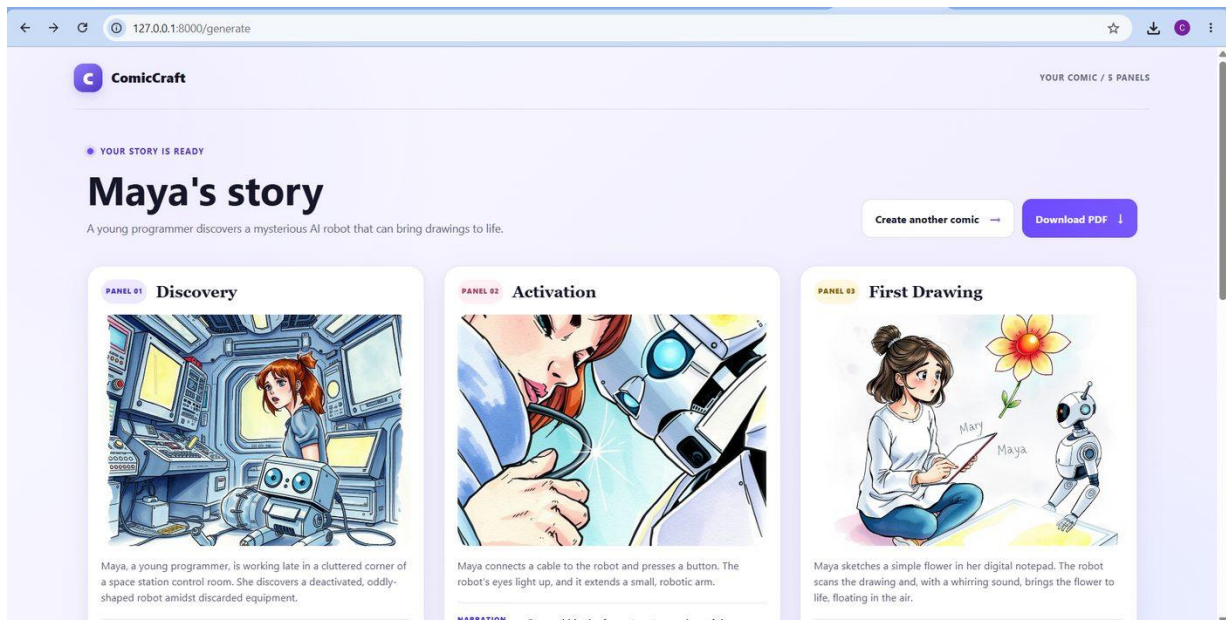
Home / Generator Page:



The screenshot shows the ComicCraft website's home page. At the top, there's a navigation bar with the ComicCraft logo and a 'Start creating' button. The main content area is divided into two sections. On the left, a large card titled 'Turn your idea into a comic.' contains a form for generating a comic. The form includes a 'Story prompt' text area with the example text 'A young programmer discovers a mysterious AI robot that can bring drawings to life.' Below this are four dropdown menus: 'Main character' (set to 'Maya'), 'Setting' (set to 'Space station'), 'Story tone' (set to 'Whimsical'), and 'Art style' (set to 'Watercolor comic'). A large purple 'Generate comic' button is at the bottom of this form. On the right, a preview card titled 'AI POWERED' and '5 PANELS' shows a sample comic panel with a sun and a character, and a 'YOUR STORY' section with the text 'One idea. Five big moments.' and five numbered buttons (01 to 05).

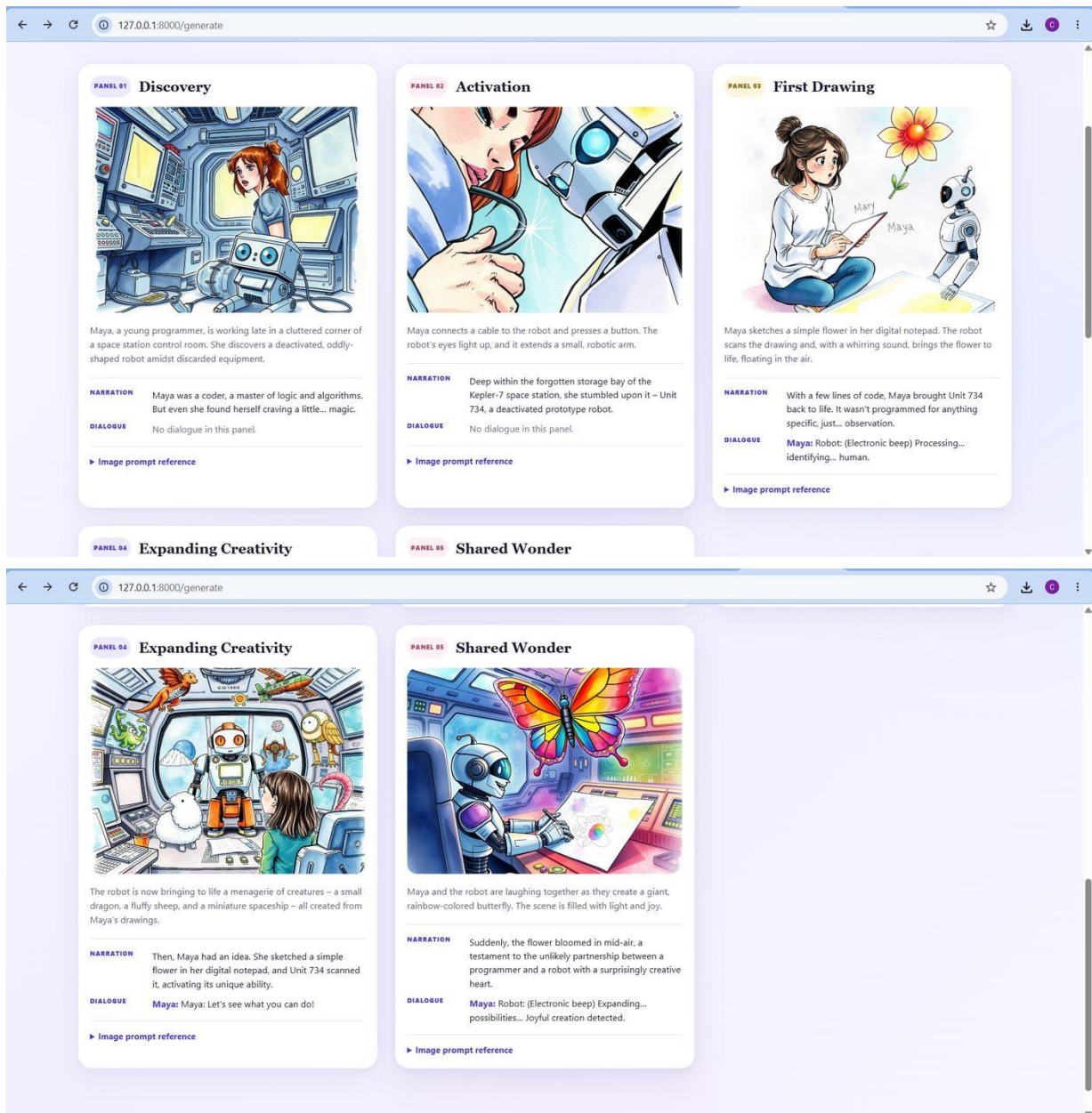
Description: The home page is the landing page and the story form in one. A header shows the ComicCraft logo and a Start creating button. The main card holds the story form with the story prompt, main character, setting, story tone and art style, followed by the Generate comic button. A preview card on the right explains the five-panel result.

Comic Result Page:



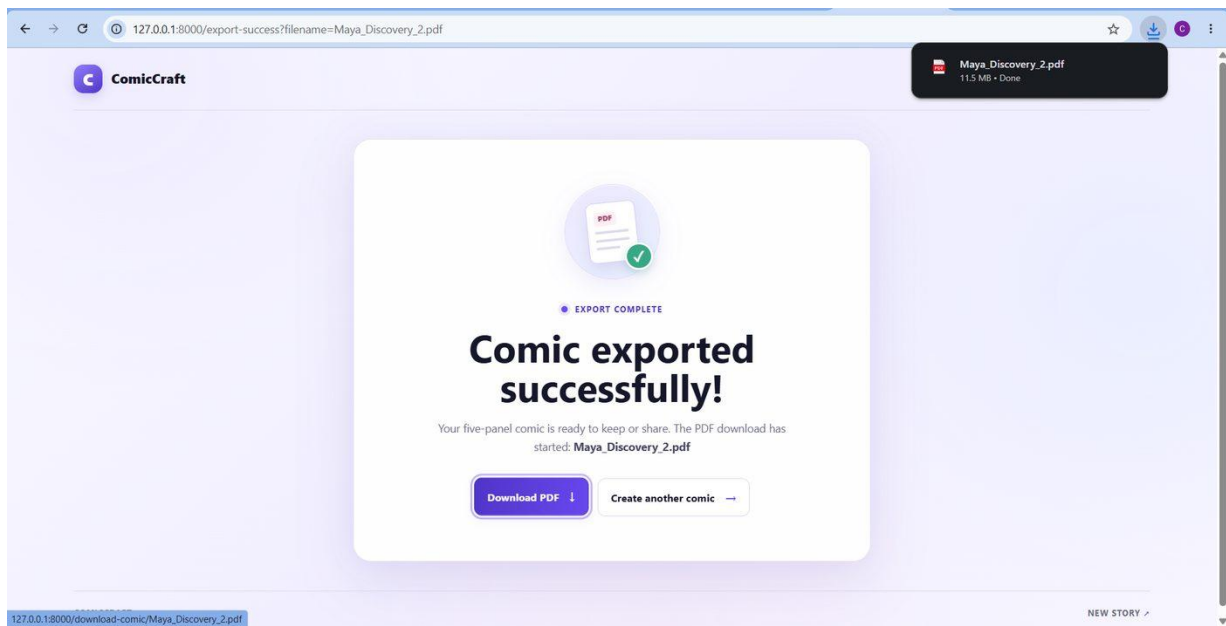
The screenshot shows the ComicCraft website's result page. At the top, there's a navigation bar with the ComicCraft logo and a 'YOUR COMIC / 5 PANELS' label. The main content area is divided into two sections. On the left, a large card titled 'Maya's story' contains the story title and the original prompt 'A young programmer discovers a mysterious AI robot that can bring drawings to life.' Below this are five comic panels, each with a title, a generated image, and a short scene description. The panels are: 'PANEL 01: Discovery' (Maya in a space station control room), 'PANEL 02: Activation' (Maya connecting a cable to the robot), 'PANEL 03: First Drawing' (Maya sketching a flower), 'PANEL 04: (Title obscured)' (Maya and the robot), and 'PANEL 05: (Title obscured)' (Maya and the robot). At the bottom of the panels, there's a 'NARRATION' section with the text 'Deen within the forgotten storage bay of the'. On the right, there are two buttons: 'Create another comic' and 'Download PDF'.

Description: After generation, the result page shows the story title and the original prompt, with Create another comic and Download PDF buttons at the top. The five panels are shown as cards in a grid. Each card has the panel number and title, the generated image and a short scene description.



Description: Below each image, the card shows the narration and the dialogue for that panel, and an expandable image prompt reference that shows the prompt used to create the picture. Panels without speech show that there is no dialogue.

Export Success Page:



Description: After the user clicks Download PDF, this page confirms that the comic was exported successfully and shows the PDF file name. The download starts automatically, and the page offers a Download PDF button to download again and a Create another comic button to start a new story.

Conclusion:

ComicCraft is a generative AI platform that turns a short story idea into a complete five-panel illustrated comic. It uses Google's Gemini models to write the story, including narration and dialogue, and to create an image for every panel in the art style the user chooses, then exports the result as a PDF. The project, which included model selection, backend and frontend development and local deployment, shows how generative AI can make storytelling and comic creation accessible to people without drawing skills. Looking ahead, ComicCraft offers opportunities for expansion, such as supporting more panels, adding more art styles and languages, keeping characters consistent across panels, and adding user accounts to save past comics. By continuously refining the prompts and improving the interface, the platform can grow from a simple comic maker into a complete storytelling tool for students, teachers and creators.